ТЗ: прототип агента «О!дин запрос»

Oct 23, 2026 · @Timur

Цель и рамки MVP

Собираем работающий прототип, где можно пройти путь от одного сообщения до имитированной покупки. Ядро продукта: лента роликов отелей с ИИ-озвучкой, где свайпы заменяют длинную анкету, а агент объясняет каждый выбор.

Делаем в MVP

- Разбор запроса на естественном языке в профиль поездки
- Ленту роликов с ИИ-озвучкой цены, условий и выжимки отзывов
- Свайпы, которые обновляют веса предпочтений
- Подборку с объяснением каждого варианта и правкой логики
- Пакет: отель + билеты (авиа или ЖД) + трансфер, с проверкой состыковок
- Страницу доуточнения и имитацию оплаты

Не делаем в MVP: работу после покупки, память между сессиями, реальную оплату, экскурсии и активности. Если останется время, первым берём бота или Telegram Mini App.

Готово к защите, когда:

- [ ] Любой из 20 тестовых запросов проходит до оплаты без падений
- [ ] Незнакомый запрос жюри доходит до первого ролика быстрее 10 секунд
- [ ] Изменение условия («на два дня позже») пересобирает пакет, а не начинает поиск заново
- [ ] На демо честно сказано, какие данные моковые

Пользовательский путь

У пути два входа, а дальше он один. Полная схема со всеми развилками лежит в BPMN прототипа, исходники `odin-zapros-mvp.bpmn` и `odin-zapros-content.bpmn` лежат в файлах проекта.

- **Сценарий 1, основной.** Человек коротко пишет запрос. Агент вытаскивает жёсткие параметры (даты, бюджет, гости) в фильтры и строит ленту. Второстепенное (линия, питание, звёзды, тип отдыха) понимаем по свайпам.

- **Сценарий 2, известно всё.** Если из сообщения удалось вытащить 100% параметров, вплоть до отеля и категории номера, лента пропускается и агент сразу собирает пакет.

Общий путь после ленты:

1. Каждые 3–5 лайков агент показывает подборку. У каждого варианта есть объяснение: почему подходит, чем жертвуем, риски.
2. Человек оформляет, смотрит ещё или правит логику («питание не важно»). Правка обновляет веса и пересобирает подборку.
3. Агент параллельно ищет билеты и трансфер и собирает пакет с объяснением.
4. На странице доуточнения человек проверяет несколько ключевых полей и отвечает на «что-нибудь ещё?» или пропускает.
5. Пересчёт цены и проверка состыковок. Конфликт возвращает на страницу доуточнения.
6. Имитация оплаты, итоговый план и ваучеры.

Архитектура и стек

Система состоит из четырёх слоёв: приложение, бэкенд с агентом, внешние сервисы и данные. LLM берёт на себя только язык, а ранжирование, проверку состыковок и оплату делает обычный код. Так надёжнее и быстрее на запросах жюри.



архитектура прототипа · 4 слоя

Слой	Технологии	Почему
Приложение	React или Next.js, Swiper для вертикальной ленты, HTML5 video + audio	быстро, много готовых компонентов; потом можно обернуть в Telegram Mini App
Бэкенд	Python, FastAPI, SQLAlchemy	один язык с агентом и скриптами данных
Агент	LangGraph + LLM с вызовом функций и structured output	граф повторяет BPMN, легко показать жюри
Синтез речи	Yandex SpeechKit, ElevenLabs или бесплатный Silero	аудио играет поверх видео, склеивать не нужно
Данные	PostgreSQL (на хакатоне хватит SQLite), файлы роликов в объектном хранилище или статике	просто и бесплатно
Деплой	Docker, фронт на Vercel, бэк на Render или любом VPS	ссылка для жюри работает с телефона

Агент

Агент делаем на LangGraph: граф шагов повторяет BPMN, каждый блок агента становится узлом, развилка становится условным переходом. Состояние графа это профиль поездки плюс подборка и корзина, оно хранится в базе по `session_id`. LLM вызываем только там, где нужен язык. Всё остальное (фильтры, ранжирование, состыковки, оплата) делает обычный код.

Профиль поездки

Это главный контракт между LLM, бэком и фронтом. Поле `source` показывает, откуда значение: из текста, из свайпов или по умолчанию. На странице доуточнения выведенные значения подсвечиваем, чтобы человек их проверил.

```
{
  "session_id": "s_123",
  "hard": {
    "date_from": "2026-11-10", "date_to": "2026-11-17", "flexible_days": 2,
    "budget_total_rub": 150000,
    "adults": 2, "children_ages": [5],
    "origin_city": "Москва",
    "destination": null, "hotel_id": null, "room_category": null,
    "constraints": ["без перелёта дольше 5 ч", "с собакой"]
  },
  "weights": {
    "beach_line_1": 0.6, "all_inclusive": 0.2, "stars_5": 0.1,
    "kids_club": 0.8, "active": -0.3, "quiet": 0.5, "free_cancel": 0.4
  },
  "source": { "budget_total_rub": "text", "kids_club": "swipes",
"origin_city": "default" },
  "liked": ["v_017", "v_042"], "skipped": ["v_003"],
  "cart": null
}
```

Где работает LLM

Узел	Вход	Выход
`parse_request`	текст пользователя	`hard` + стартовые `weights` в JSON (structured output) + флаг «известно всё»

Узел	Вход	Выход
`voiceover`	карточка отеля, цена на даты, выжимка отзывов, профиль	текст на 15–20 секунд (до 50 слов)
`explain`	вариант, веса, сравнение с другими вариантами	`why`, `tradeoffs`, `risks`, по 1–2 предложения
`apply_correction`	фраза вроде «питание не важно»	дельта к `weights` или `hard`
`anything_else`	свободный текст со страницы доуточнения	дельта к профилю и список затронутых компонентов пакета

Пересборка не с нуля: дельта говорит, что изменилось, и перезапрашиваем только затронутое. Смена дат обновляет цены и билеты, «с собакой» добавляет фильтр и пересчитывает ленту.

Инструменты агента

Функция	Вход	Выход	Источник
`search_hotels`	`hard`, `weights`, исключения	список роликов с отелями по рейтингу	наш каталог
`get_hotel_price`	`hotel_id`, категория номера, даты, гости	цена, наличие, питание, условия отмены, время заезда	мок вместо API Островка
`search_flights`	город вылета, направление, даты	рейсы: цена, время, пересадки	Travelpayouts (кэш цен)
`search_trains`	станции, дата	поезда: отправление, прибытие, время в пути + цена по типу вагона	Яндекс.Расписания + мок цен
`search_transfer`	точка прибытия, отель, время	варианты: такси, трансфер, время, цена	мок

Функция	Вход	Выход	Источник
`get_weather`	город, месяц	температура воздуха и воды, осадки	Open-Meteo
`check_connections`	пакет	список конфликтов (прилёт после последнего трансфера, заезд раньше приезда, даты не совпадают)	код, без LLM
`book`	пакет, данные гостей	номер заказа, ваучеры	мок

Лента и ранжирование

Лента ранжируется простой формулой без ML: жюри легко объяснить, команде легко отладить. Сначала жёсткие фильтры отсекают неподходящее (даты, наличие, бюджет с запасом 10%, гости, ограничения), потом оставшееся сортируется по рейтингу.

```
score(v) = \sum_i w_i \cdot f_i(v) - 0.3 \cdot sim(v, skipped) + 0.1 \cdot novelty(v)
```

- `f_i(v)` это признаки ролика от 0 до 1: первая линия, всё включено, звёзды, детский клуб, активный или спокойный отдых, бесплатная отмена.
- `w_i` это веса из профиля от −1 до 1.
- `sim` это похожесть на пропущенные ролики, `novelty` не даёт ленте зациклиться на одном направлении. Коэффициенты 0.3 и 0.1 стартовые, подбираем на тестовых запросах.

Правила свайпов

Событие	Что меняется
Свайп вправо	Ролик в `liked`; каждый вес сдвигается к признаку ролика: w = w + 0.2 × (f − 0.5)
Свайп влево	Ролик в `skipped`; сдвиг в обратную сторону с шагом 0.1

Событие	Что меняется
Досмотрел до конца без свайпа	Слабый плюс, шаг 0.05 (опционально)
3–5 лайков с последней подборки	Показать подборку с объяснениями

Следующие 3 ролика предзагружаем вместе с озвучкой, чтобы свайп не ждал LLM и синтез речи.

Данные и контент

Для демо хватит 30–50 отелей в 3–5 направлениях. Направления берём так, чтобы закрыть типовые запросы жюри: море в России (Сочи), тёплое море зимой (Турция, Египет или ОАЭ), город на выходные (Петербург, Казань), природа (Алтай или Карелия). Список финализируем после тестовых запросов.

Данные	Объём на демо	Источник	Реальные или мок
Отели: описание, координаты, звёзды, линия, питание, категории номеров	30–50	собираем вручную из открытых источников	реальные отели, данные частично синтетические
Цены и наличие номеров	на любые даты	генератор: базовая цена × сезонный коэффициент × день недели	мок (вместо API Островка)
Ролики	2–3 на отель	промо отелей или ИИ-видео по фото	ИИ-ролики помечены в кадре
Отзывы	20–50 на отель	открытые датасеты отзывов об отелях, иначе синтетика	выжимку делает LLM заранее
Авиабилеты	по запросу	Travelpayouts Data API	реальный кэш цен
ЖД	по запросу	API Яндекс.Расписаний	расписание реальное, цены мок
Трансферы	таблица на направление	расстояние по координатам × тариф	мок

Данные	Объём на демо	Источник	Реальные или мок
Погода	по запросу	Open-Meteo	реальные

Карточка ролика. У каждого ролика: `video_id`, `hotel_id`, `room_category`, `source` (`partner` или `ai_from_photos`), ссылка на файл, длительность и признаки `f_i`. Признаки размечаем полуавтоматически: LLM предлагает по описанию отеля, человек проверяет. Цены в ролик не вшиваем, они приходят в озвучке в момент показа.

ИИ-видео по фото. Делаем заранее, не в момент запроса. Варианты по цене и времени: image-to-video сервисы (Kling, Runway, Luma) или бесплатное слайдшоу с плавным наездом через ffmpeg. Нельзя дорисовывать то, чего нет на фото.

Экраны фронта

Всего шесть экранов под ширину телефона (390–430 px). На демо показываем с телефона или в режиме телефона в браузере.

Экран	Что на нём	Ключевое поведение
1. Запрос	Поле ввода, 3–4 примера запроса чипами	После отправки показываем, что агент понял: чипы с датами, бюджетом, гостями
2. Лента	Полноэкранное вертикальное видео, озвучка, субтитры, плашка с ценой и счётчик лайков	Свайп вправо и влево, кнопки дублируют свайп. Метка «ИИ по фото отеля», где нужно
3. Подборка	Карточки понравившихся вариантов: почему подходит, чем жертвуем, риски	Кнопки «Оформить», «Смотреть ещё» и поле «Поправить логику». Видно, какие приоритеты агент вывел из свайпов
4. Доуточнение	Пакет: отель, билеты, трансфер, итого. 4–5 ключевых полей: гости, даты, город вылета, рейс	Поля уже заполнены, выведенные подсвечены. Рейс можно поменять на дешёвый или быстрый. Вопрос «Что-нибудь ещё?» с кнопкой «Пропустить»
5. Оплата	Итог, данные гостей, кнопка «Оплатить»	Прямо на экране подписано, что оплата демонстрационная

Экран	Что на нём	Ключевое поведение
6. Готово	План поездки по часам: вылет, прилёт, трансфер, заезд. Ваучеры	—

В любой момент есть поле для текста («а если на два дня позже?»), оно идёт в `apply_correction`.

API бэкенда

Фронт говорит с бэком через REST на FastAPI. Долгие ответы агента (объяснения, сборка пакета) стримим через SSE, чтобы человек не смотрел на пустой экран. Договор фиксируем в первый день, фронт до готовности бэка работает на фейковых ответах.

Метод	Что делает	Ответ
`POST /session`	текст запроса → разбор	`session_id`, профиль, `mode`: `feed` или `checkout`
`GET /session/{id}/feed?n=3`	следующие ролики	ролики с аудио озвучки, текстом и ценой
`POST /session/{id}/swipe`	`video_id`, `direction`	обновлённые веса, флаг «показать подборку»
`GET /session/{id}/shortlist`	подборка	варианты с `why`, `tradeoffs`, `risks` (SSE)
`POST /session/{id}/correct`	текст правки	новая подборка или новый пакет
`POST /session/{id}/package`	выбранный отель	пакет: отель, билеты, трансфер, объяснение, конфликты (SSE)
`PATCH /session/{id}/package`	правки со страницы доуточнения	пересчитанный пакет
`POST /session/{id}/book`	данные гостей	номер заказа, план, ваучеры
`GET /metrics`	для нас и жюри	сводка по метрикам из раздела ниже

Аудио озвучки кэшируем по ключу «ролик + даты + гости»: для демо-запросов оно будет готово заранее.

Метрики и тестовые запросы

Собираем 20 тестовых запросов и гоняем их скриптом после каждого большого изменения. Свайпы в скрипте имитируем правилом «лайк, если ролик подходит под скрытые предпочтения теста». Так метрики доп. задачи кейса получатся сами собой.

Метрика	Как считаем	Цель на демо
Доля сценариев до оплаты	дошли до `book` / всего запросов	20 из 20
Время до первого ролика	от отправки запроса до старта видео	меньше 10 секунд
Вопросов к пользователю	полей, которые пришлось спросить или исправить на странице доуточнения	не больше 3
Свайпов до подборки	среднее по тестам	ориентир 10–15
Точность разбора	поля `hard`, совпавшие с эталоном теста	90% и выше
Ошибки в состыковках	конфликты, дошедшие до оплаты	0
Пересборка не с нуля	при правке перезапрошены только затронутые компоненты	во всех тестах с правкой

Примеры тестовых запросов

1. «Хочу куда-нибудь к тёплому морю в ноябре, недорого, с ребёнком, без долгих перелётов» (пример из кейса)
2. «Выходные в Питере вдвоём, из Москвы на Сапсане, 14–16 ноября»
3. «Отель X в Сочи, номер делюкс с видом на море, 3–10 декабря, двое взрослых» (сценарий 2, без ленты)
4. После подборки: «а если на два дня позже?», «бюджет меньше», «с нами едет собака» (примеры из кейса)

Остальные 16 дописывает продакт, смешивая размытые, точные и каверзные запросы (нет вариантов в бюджет, противоречивые условия).

Роли, план и открытые вопросы

Разбивка по ролям, имена впишем, когда распределим. Если людей меньше, «Агент» забирает бэкендер.

Роль	Зона ответственности	Первая задача
Продакт / проджект (Тимур)	скоуп, данные и контент, тестовые запросы, метрики, сценарий демо и питч	список направлений и отелей, 20 тестовых запросов
Фронтенд	6 экранов, лента с видео и аудио, свайпы	лента на фейковых данных
Бэкенд	FastAPI, база, инструменты, моки цен и трансферов, интеграции с Travelpayouts, Яндекс.Расписаниями, Open-Meteo	схема базы и договор API
Агент / LLM	граф LangGraph, промпты, озвучка, ранжирование	`parse_request` на 20 тестовых запросах

План по этапам. Каждый этап заканчивается рабочим сквозным путём, чтобы в любой момент было что показать.

1. **Каркас.** Договор API и профиля, схема базы, 10 отелей в одном направлении, лента на фейках, тестовые запросы.
2. **Сквозной путь.** Разбор запроса, лента с ранжированием, подборка, пакет только с отелем, мок-оплата.
3. **Поездка целиком.** Билеты, трансфер, проверка состыковок, страница доуточнения.
4. **Объяснения и пересборка.** `explain`, `apply_correction`, озвучка с отзывами, правка логики.
5. **Контент и полировка.** Все 30–50 отелей, ИИ-ролики, прогон метрик, скорость.
6. **Демо.** Сценарий защиты, запасное видео на случай падения сети, репетиция на незнакомых запросах.

Открытые вопросы

- [ ] Дата защиты и состав команды: от них зависит, сколько дней на этап
- [ ] Какую LLM берём и есть ли бюджет на API (если нет, смотрим YandexGPT или GigaChat с бесплатными лимитами)

- [ ] Какой синтез речи: платный с живым голосом или бесплатный Silero
- [ ] Проверить условия и лимиты API Travelpayouts и Яндекс.Расписаний до старта разработки
- [ ] Где берём реальные фото отелей и можно ли их использовать на демо